Clase 247 — Seguridad de APIs en el ciclo de desarrollo

> Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: OWASP API Security Top 10 (2023) y OWASP ASVS v4 ⏱ Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Integrar la seguridad de APIs en cada fase del desarrollo: diseñar contratos seguros (OpenAPI), prevenir las vulnerabilidades del OWASP API Security Top 10 (especialmente los fallos de autorización BOLA/BFLA, que dominan los incidentes reales), y automatizar la validación de contrato y el fuzzing en el pipeline. Las APIs son hoy la principal superficie de ataque de las aplicaciones; asegurarlas "por diseño" y con tests automatizados es clave.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Reconocer** las categorías del OWASP API Security Top 10 y sus causas raíz.
2. **Diseñar** un contrato OpenAPI con autenticación, autorización y validación explícitas.
3. **Detectar** fallos de autorización a nivel de objeto (BOLA) y de función (BFLA).
4. **Automatizar** validación de contrato y fuzzing de la API en CI.
5. **Aplicar** controles: rate limiting, validación de esquema, mínima exposición de datos.

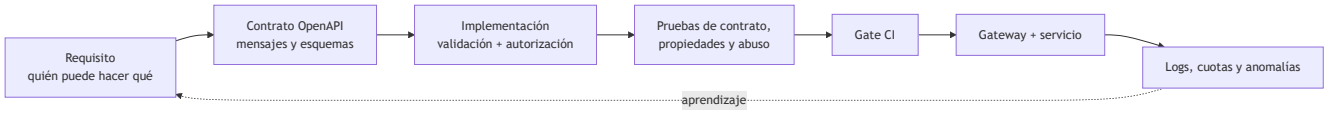
🗺 Temas

#	Tema	Por qué importa
1	OWASP API Security Top 10 (2023)	Taxonomía específica de APIs
2	BOLA (API1) y BFLA (API5)	Los fallos de autorización dominan los incidentes
3	Contrato OpenAPI como fuente de verdad	Diseñar seguridad, no parchearla
4	Validación de esquema y entrada	Rechazar lo que no cumple el contrato
5	Autenticación y gestión de tokens	JWT, scopes, expiración
6	Rate limiting y consumo de recursos	API4: abuso y DoS
7	Fuzzing y testing de contrato en CI	Automatizar la verificación

🧠 Explicación en profundidad

El contrato describe mensajes; la autorización protege objetos y operaciones

Una API segura comienza definiendo activos, consumidores, operaciones y propiedades de autorización. OpenAPI puede especificar rutas, esquemas y mecanismos de autenticación, pero no demuestra que el usuario A deba leer el pedido 42. Esa relación pertenece a la lógica de negocio y debe expresarse mediante requisitos, decisiones centralizadas y pruebas negativas.



El contrato permite *linting*, generación de casos y detección de cambios incompatibles. Los esquemas restringen forma, tamaño y tipo; las validaciones semánticas comprueban reglas como fechas o transiciones. En el servicio, autenticación establece una identidad y autorización decide cada objeto, campo y función. Un gateway puede verificar token y cuota, pero no conoce por sí solo todas las relaciones del dominio.

Diseñar pruebas desde propiedades

La suite combina casos válidos con abuso: identificadores de otro usuario, rol insuficiente, campos adicionales, listas masivas, reintentos, estados inválidos y funciones administrativas. Para BOLA se crean dos identidades y objetos diferenciados; se cambia solo el identificador y se exige una denegación sin filtrar existencia. Para consumo de recursos se prueba paginación, límites, coste y cancelación, no solo peticiones por segundo.

El *fuzzing* guiado por esquema explora combinaciones que el equipo no escribió, pero su calidad depende del contrato y del oráculo. Una respuesta `500` es un indicio; se investiga el efecto y la reproducibilidad. En APIs GraphQL, gRPC y asíncronas cambian las superficies, aunque permanecen las preguntas de identidad, límites y validación.

Cambios y observabilidad

Una política de versión protege consumidores, pero mantener una versión vulnerable indefinidamente tampoco es seguro. Se identifican clientes, se publica deprecación, se mide uso y se retira con plan. La telemetría conserva actor, operación, objeto lógico, decisión y correlación sin registrar tokens ni datos sensibles completos.

Caso razonado: autorización solo en la interfaz

La UI oculta «exportar todos» a usuarios normales, pero la API acepta la operación si se conoce la ruta. El contrato documenta el endpoint sin su requisito de rol y las pruebas solo usan administrador. Se agrega una política de autorización en servicio, casos negativos por rol, una regla de revisión para operaciones sensibles y alerta de intentos denegados. Ocultar un botón era experiencia de usuario, no control de acceso.

📓 Glosario operativo

Término	Definición útil
Contrato	Descripción interoperable de operaciones y mensajes; no prueba autorización.
BOLA	Acceso indebido a objetos por faltar una comprobación contextual.
Mass assignment	Modificación de campos internos al enlazar entrada sin lista permitida.
Rate limit	Restricción de frecuencia; debe considerar identidad, operación y coste.
Prueba negativa	Caso que exige rechazar una acción inválida o no autorizada.

✅ Criterio de dominio

El alumno domina la clase si transforma propiedades de negocio en contrato, implementación y pruebas negativas; diferencia gateway de autorización de dominio; controla recursos; y conserva evidencia operativa sin filtrar secretos.

Cómo lo nombra el catálogo CWE y qué cambia al priorizar

El caso razonado de la clase, «autorización solo en la interfaz», tiene nombre en el catálogo: `CWE-602` (*aplicación en el cliente de una seguridad que corresponde al servidor*), una entrada de nivel *Class* con uso «permitido con revisión», hija de `CWE-693` (fallo de un mecanismo de protección). Es la causa de que un contrato OpenAPI que describe campos y tipos no proteja nada por sí mismo: el contrato describe mensajes, y la autorización es otra propiedad. Cuando el servidor sí omite la comprobación, la debilidad concreta es `CWE-862` (falta de autorización) o `CWE-863` (autorización incorrecta), y si la clave del objeto la controla el cliente, `CWE-639`.

Las tres son las que la clase 364 practica con una matriz de acceso: una prueba que recorre todas las parejas usuario–recurso y comprueba la política, en lugar de un solo caso feliz. Esa es la forma de volver «probar desde propiedades» (el tema de esta clase) algo ejecutable: `labs/appsec-code/practicas/autorizacion.py` y sus pruebas muestran la versión vulnerable que viola la política, la que tiene la comprobación equivocada y la correcta.

📖 Definiciones y características

- **BOLA (Broken Object Level Authorization)**: acceder a objetos de otros usuarios cambiando un ID. *Característica*: API1, la vulnerabilidad más común; se previene autorizando por objeto en cada request.
- **BFLA (Broken Function Level Authorization)**: invocar funciones/roles no permitidos (p. ej. endpoints de admin). *Característica*: API5; se previene con control de acceso por función y deny-by-default.
- **Contrato OpenAPI**: especificación formal de la API. *Característica*: fuente de verdad para validar requests/responses y generar tests.
- **Validación de esquema**: rechazar payloads que no cumplen el contrato. *Característica*: reduce inyecciones y mass assignment.
- **Mass assignment (API6)**: vincular campos del payload a propiedades internas sin filtrar. *Característica*: se previene con allowlist de campos.

- **Rate limiting**: limitar peticiones por cliente/tiempo. *Característica*: mitiga abuso, fuerza bruta y DoS (API4).

🧰 Herramientas y preparación

- **OpenAPI (Swagger)** para el contrato; **Spectral** para lintarlo.
- **Schemathesis** o **RESTler** para fuzzing basado en contrato.
- **OWASP ZAP** con importación de OpenAPI para escaneo dirigido.
- Una API de práctica propia (p. ej. **crAPI** o **VAmPI**, APIs vulnerables de laboratorio).

```
pip install schemathesis
# Fuzzing dirigido por el contrato OpenAPI:
schemathesis run http://localhost:8000/openapi.json --checks all
```

> Nota ética: crAPI, VAmPI y similares son APIs deliberadamente vulnerables para practicar. No pruebes BOLA/BFLA ni fuzzing contra APIs de terceros sin autorización explícita.

🧪 Laboratorio guiado

1. **Levanta una API vulnerable de práctica** (crAPI o VAmPI) en local.
2. **Explora BOLA**. Autentícate como usuario A, pide tu recurso (`GET /users/{idA}/profile`), luego cambia el id por el de otro usuario. Si respondes con datos ajenos, es BOLA. Documenta la causa: falta de check de propiedad.
3. **Explora BFLA**. Como usuario normal, intenta llamar un endpoint de admin (`POST /admin/...`). Si funciona, es BFLA.
4. **Diseña el contrato seguro**. Escribe/ajusta el OpenAPI: define `securitySchemes` (bearer JWT), scopes por endpoint, esquemas de request con `required` y `additionalProperties: false` para evitar mass assignment.
5. **Lintéalo con Spectral**:

```
spectral lint openapi.yaml
```

6. **Fuzzing basado en contrato**:

```
schemathesis run http://localhost:8000/openapi.json --checks all --hypothesis-max-examples 20
```

Revisa violaciones de esquema, 500s y respuestas fuera de contrato. 7. **Escaneo dirigido con ZAP**. Importa el OpenAPI en ZAP y lanza un active scan enfocado en los endpoints reales. 8. **Integra en CI**. Añade jobs de lint del contrato + Schemathesis contra una instancia efímera; falla el build ante violaciones de contrato o errores de servidor.

✍️ Ejercicios

1. Identifica un BOLA en una API de práctica y describe cómo corregirlo.
2. Diseña un contrato OpenAPI con auth por JWT y scopes por endpoint.

3. Cierra un mass assignment con `additionalProperties: false` y allowlist.
4. Ejecuta Schemathesis y clasifica los hallazgos por gravedad.
5. Configura rate limiting y demuéstralo con una prueba de carga simple.
6. Integra lint de contrato + fuzzing como gate de CI.

📝 Reto verificable

Asegura una API de extremo a extremo con controles y verificación automatizada.

Criterio de aceptación: (a) el contrato OpenAPI define autenticación, autorización por endpoint y validación estricta de esquema; (b) los fallos BOLA y BFLA de la API de práctica están corregidos y verificados; (c) el pipeline lintea el contrato y ejecuta fuzzing basado en contrato que pasa; y (d) hay rate limiting activo demostrable.

⚠️ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Cambiar un ID en la URL devuelve datos de otro usuario	BOLA: falta autorización por objeto. Verifica propiedad en cada request.
Un usuario normal accede a endpoints de admin	BFLA: control por función ausente. Deny-by-default y checks por rol.
El cliente envía `is_admin: true` y funciona	Mass assignment. Usa allowlist de campos y `additionalProperties: false`.
La API devuelve todo el objeto de usuario	Exposición excesiva de datos. Serializa solo los campos necesarios.
Fuerza bruta al login sin límite	Falta rate limiting (API4). Añade límites por IP/usuario y backoff.

❓ Preguntas frecuentes

❓ **¿Por qué el Top 10 de APIs es distinto del web clásico?** Las APIs exponen lógica y objetos directamente y suelen delegar la UI al cliente; por eso los fallos de autorización (BOLA/BFLA) y de consumo de recursos dominan, más que XSS clásico.

❓ **¿Un WAF me protege de BOLA?** Casi nunca. BOLA es un fallo de lógica de autorización: el request es "válido" técnicamente. Se corrige en el código verificando la propiedad del objeto, no con firmas genéricas.

❓ **¿El contrato OpenAPI mejora la seguridad o solo la documentación?** Ambas. Como fuente de verdad permite validar entradas, generar tests/fuzzing y detectar respuestas fuera de contrato automáticamente.

❓ **¿Fuzzing de contrato reemplaza al pentest de API?** No. Encuentra desviaciones de esquema y errores, pero la lógica de autorización y las cadenas de abuso siguen necesitando análisis manual.

🔗 Referencias

- OWASP API Security Top 10 (2023) — https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- OWASP ASVS — https://owasp.org/www-project-application-security-verification-standard/
- Schemathesis — https://schemathesis.readthedocs.io/
- Spectral (OpenAPI linter) — https://stoplight.io/open-source/spectral
- OWASP crAPI — https://github.com/OWASP/crAPI

📥 Material descargable

- 📄 Guía en PDF — versión imprimible de esta clase.
- 🎞️ Presentación (PPTX) — deck para proyectar en clase.

⬅️ Clase anterior

Clase 246 — Supply chain security: SBOM y SLSA

➡️ Siguiente clase

Clase 248 — Cultura DevSecOps y security champions